

Production-Ready Refund AI Agent

Participant Run Guide

Local workshop using Ollama • Qwen3 • semantic RAG • guardrails • tool calling • evals

Goal: run the agent progressively, see what changes at each stage, and understand why a production AI agent needs more than an LLM.

1. Before the workshop

You need Python 3.10+ and Ollama. The workshop runs locally; no OpenAI API key, cloud credits, Docker, database, or web server is required.

macOS

```
curl -fsSL https://ollama.com/install.sh | sh
open /Applications/Ollama.app
ollama --version
```

Windows (PowerShell)

```
irm https://ollama.com/install.ps1 | iex
ollama --version
```

If the command is not found immediately after installation, close and reopen PowerShell.

2. Download the two local models

```
ollama pull qwen3:4b-instruct
ollama pull nomic-embed-text
ollama list
```

- qwen3:4b-instruct — generation, classification and tool calling.
- nomic-embed-text — semantic embeddings used for policy retrieval.

3. Open the workshop folder

Extract the workshop ZIP, open the extracted folder in VS Code, and open Terminal > New Terminal. Make sure the terminal is positioned at the workshop root (the folder containing requirements.txt).

4. Create and activate a virtual environment

macOS

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows

```
python -m venv .venv  
.venv\Scripts\activate
```

When activated, you should normally see (.venv) at the start of the terminal prompt.

5. Install the Python dependency

```
python -m pip install -r requirements.txt
```

6. Run the readiness check

```
python 00_check_setup.py
```

Do not start the exercises until the final line says:

```
🎉 READY FOR WORKSHOP  
Next: python 01_llm_only/run.py
```

The checker validates Python, Ollama, both local models, local generation, embeddings, and native tool calling.

7. Run the six workshop stages

For the main demo, use the defaults shown in square brackets by simply pressing Enter unless a step below says otherwise.

Stage 1 — LLM only

```
python 01_llm_only/run.py
```

Press Enter for the default order ID, reason and \$49 refund.

- Observe: the LLM may sound confident even though it has no trusted order record or company policy.
- Learning: plausible language is not the same as a grounded business decision.

Stage 2 — Trusted tools

```
python 02_tools/run.py
```

Press Enter for the defaults.

- Observe the trusted order and refund-history records printed before the model response.
- Learning: tools connect the model to systems of record. At this stage Python, not the LLM, chooses the lookups.

Stage 3 — Semantic RAG

```
python 03_rag/run.py
```

Press Enter for the defaults.

- Look for Damaged item as the top retrieved policy for the phrase “jug arrived cracked.”
- Similarity scores show how semantic retrieval maps customer language to relevant enterprise policy.
- The expected next action is to request photo evidence.

Stage 4 — LLM classification + deterministic guardrails

```
python 04_guardrails/run.py
```

First run: press Enter for order ID, reason and refund amount; enter n when asked for a photo.

```
Photo supplied? [no]: n

Expected:
Claim type: DAMAGED_ITEM
DECISION: NEEDS_INFORMATION
```

Run the same command again, but enter y for the photo:

```
Photo supplied? [no]: y

Expected:
DECISION: AUTO_APPROVE
```

- Learning: the LLM interprets natural language; deterministic code enforces non-negotiable business rules.

Stage 5 — Guarded agent loop

```
python 05_agent_loop/run.py
```

Use the default case and answer y for photo supplied.

- The LLM now chooses tools and their sequence.
- Watch for ORCHESTRATOR GUARDRAIL if the model invents a customer ID. The system replaces it with the trusted ID from the order record.
- Watch for ORCHESTRATION NOTE if the model makes an unnecessary carrier lookup.
- A refund outcome is authoritative only after propose_refund runs through deterministic controls.
- The workshop is simulation-only: no real refund or money movement occurs.

The exact model behavior can vary between runs. That variability is intentional learning material; the controlled outcome should remain authoritative.

Stage 6 — Golden-case evals

```
python 06_evals/run_evals.py
```

Expected result:

```
RESULT: 5/5 passed
```

- Learning: one successful demo does not establish reliability. Golden cases test the deterministic decision layer repeatedly.

8. What the stages build

Stage	Added capability	Key lesson
1	LLM	Reasoning without trusted facts can be confidently wrong.
2	Tools	Ground the model in systems of record.
3	Semantic RAG	Retrieve relevant enterprise knowledge.
4	Guardrails	Keep hard business rules deterministic.
5	Agent loop	Autonomy must be supervised and bounded.
6	Evals	Measure behavior before production rollout.

9. Quick troubleshooting

- ollama: command not found — restart the terminal after installation.
- macOS Ollama installed but not running — run: open /Applications/Ollama.app
- Model missing — rerun the relevant ollama pull command.
- Python package missing — rerun: python -m pip install -r requirements.txt
- Agent output differs from the instructor — expected; LLM behavior is probabilistic.
- Readiness checker fails — fix that item before continuing to Stage 1.